

GROKING: 소규모 알고리즘 데이터셋에서 과적합을 넘어서 일반화

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin - OpenAI
Vedant Misra - Google

한국어 번역본 | 원문: arXiv:2201.02177v1 (2022년 1월)

초록

본 논문은 알고리즘적으로 생성된 소규모 데이터셋에서 신경망의 일반화를 연구한다. 이 환경에서는 데이터 효율, 암기, 일반화, 학습 속도에 관한 질문을 매우 세밀하게 살필 수 있다. 일부 상황에서 신경망은 데이터 속 패턴을 “그로킹(grokking)”하는 과정을 거쳐, 일반화 성능이 무작위 추측 수준에서 완전한 일반화로 향상됨을 보인다. 이러한 일반화의 향상은 과적합이 일어난 시점보다 한참 뒤에 발생할 수 있다. 또한 데이터셋 크기에 따른 일반화를 조사한 결과, 데이터셋이 작을수록 일반화에 도달하기 위해 더 많은 최적화가 필요했다. 우리는 이 데이터셋들이 유한한 훈련 데이터셋의 암기를 넘어서는 과매개변수화 신경망의 일반화라는, 아직 충분히 이해되지 않은 딥러닝의 측면을 연구하기에 비옥한 기반을 제공한다고 주장한다.

1 서론

과매개변수화 신경망의 일반화는 고전적 학습 이론에서 비롯된 직관에 어긋나기 때문에 오랫동안 머신러닝 공동체의 관심사였다. 본 논문은 소규모 알고리즘 생성 데이터셋으로 신경망을 훈련할 때 훈련 집합 성능과 뚜렷이 분리된 특이한 일반화 양상이, 자연 데이터에서 유래한 데이터셋보다 훨씬 더 두드러지게 재현될 수 있음을 보인다(그림 1 왼쪽). 이러한 실험은 단일 GPU에서도 빠르게 재현할 수 있으므로 일반화 이론을 시험하는 데 편리한 테스트베드가 된다.

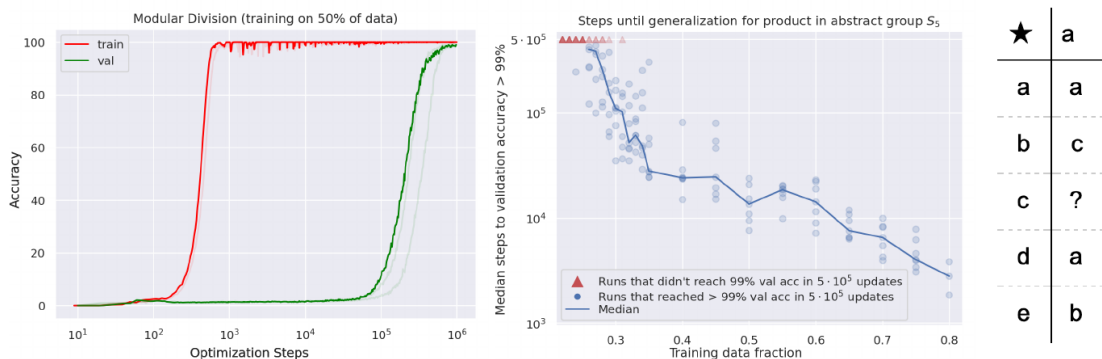


그림 1: 왼쪽. 그로킹: 알고리즘 데이터셋에서 과적합이 발생한 한참 뒤에 일반화가 나타나는 극적인 사례. 훈련 집합에는 mod 97 나눗셈 이항 연산의 데이터 중 50%를 사용했다. 97개의 잔여류는 각각 독립된 기호로 제시된다. 빨간 곡선은 훈련 정확도, 초록 곡선은 검증 정확도이다. 훈련 정확도는 최적화 10^3 단계 이전에 거의 완벽해지지만, 검증 정확도가 그 수준에 도달하려면 약 10^6 단계가 필요하며, 10^5 단계 전까지는 일반화의 징후가 거의 없다. 가운데. 검증 정확도 99%에 도달하는 데 필요한 훈련 시간은 훈련 데이터 비율이 감소할수록 빠르게 증가한다. 오른쪽. 작은 이항 연산표의 예시로, 빠진 원소를 추측해 볼 수 있다.

* Vedant Misra는 이 연구 당시 OpenAI에 소속되어 있었다.

우리가 고려하는 데이터셋은 $a \circ b = c$ 꼴의 이항 연산표이며, 여기서 a, b, c 는 내부 구조가 없는 이산 기호이고 $\circ$ 는 이항 연산이다. 이항 연산의 예로 덧셈, 순열의 합성, 이변수 다항식이 있다. 가능한 모든 식 중 일부만을 이용해 신경망을 훈련하는 일은 이항 연산표의 빈칸을 채우는 것과 같으며, 스도쿠를 푸는 것과 유사하다. 그림 1 오른쪽에 한 예가 제시되어 있다. 식에 등장하는 서로 다른 모든 원소 a, b, c 에는 서로 다른 추상 기호를 사용하므로, 네트워크는 원소의 내부 구조를 알지 못하며 다른 원소와의 상호작용만으로 그 성질을 학습해야 한다. 예를 들어 네트워크는 십진 표기 숫자나 한 줄 표기 방식의 순열을 보지 못한다.

본 논문의 기여는 다음과 같다.

- 신경망이 다양한 이항 연산표의 빈칸으로 일반화할 수 있음을 보인다.
- 심한 과적합이 발생한 한참 뒤 검증 정확도가 때때로 무작위 수준에서 완전한 일반화 쪽으로 갑자기 상승함을 보인다. 우리는 이 현상을 “그로킹”이라고 부른다.
- 다양한 이항 연산에 대한 데이터 효율 곡선을 제시한다.
- 일반화에 필요한 최적화량이 데이터셋 크기가 감소할수록 빠르게 증가함을 실증적으로 보인다.

- 데이터 효율에 미치는 영향을 측정하기 위해 여러 최적화 세부 사항을 비교하며, 가중치 감쇠가 연구한 과제에서 일반화 개선에 특히 효과적임을 확인한다.
- 네트워크가 학습한 기호 임베딩을 시각화하고, 이 임베딩이 때때로 기호가 나타내는 수학적 대상의 인식 가능한 구조를 드러냄을 보인다.

2 방법

모든 실험에서는 $a \circ b = c$ 꼴의 식으로 구성된 데이터셋에 작은 Transformer를 훈련했다. “a”, “o”, “b”, “=”, “c”는 각각 독립된 토큰이다. 연구한 연산, 아키텍처, 훈련 하이퍼파라미터 및 토큰화의 세부 내용은 부록 A.1에 제시한다.

3 실험

3.1 과적합을 넘어선 일반화

딥러닝 실무자들은 검증 손실이 더 이상 감소하지 않은 뒤에도 검증 정확도가 조금 개선되는 현상에는 익숙하다. 검증 손실의 이중 하강(double descent)은 일부 상황에서 보고되었지만, 실무자 사이에서는 흔하지 않은 현상으로 여겨진다(Nakkiran et al., 2019; Belkin et al., 2018; d’Ascoli et al., 2020). 우리가 연구한 소규모 알고리즘 데이터셋에서는 초기 과적합 뒤의 일반화 개선이 여러 모델, 옵티마이저, 데이터셋 크기에서 나타나며, 어떤 경우에는 매우 극적이다. mod 나눗셈의 전형적 사례가 그림 1에 있다. 훈련 정확도가 최적 수준에 가까워지는 데 필요한 단계보다 1,000배 더 많은 최적화가 진행된 뒤에야 검증 정확도가 무작위 수준을 넘어 상승하기 시작한다. 그림 4에는 훈련/검증 손실도 함께 도시되어 있으며, 검증 손실의 이중 하강을 확인할 수 있다.

이러한 거동은 주어진 최적화 예산 안에서 네트워크가 일반화할 수 있는 최소 데이터셋 크기에 가까운 경우, 모든 이항 연산에서 전형적으로 관찰되었다. 데이터셋이 더 크면 훈련 곡선과 검증 곡선이 대체로 더 가깝게 함께 움직인다.

3.1.1 학습 시간 곡선

일반적인 지도학습 문제에서는 최적화 과정이 훈련 데이터를 보간할 수 있을 때, 훈련 데이터 양을 줄이면 수렴한 일반화 성능이 감소한다. 반면 우리의 설정에서는 다른 현상을 관찰한다. 일정 범위의 훈련 데이터셋 크기에서는 수렴 성능이 100%로 유지되지만, 그 성능을 달성하기 위해 필요한 최적화 시간은 데이터셋 크기가 작아질수록 급격히 증가한다.



그림 2: 왼쪽. 추상 군 S5에서 곱셈을 학습하는 문제에서, 서로 다른 최적화 알고리즘은 10^4 단계의 최적화 예산 안에서 서로 다른 정도의 일반화를 낳는다. 가중치 감쇠가 일반화를 가장 크게 개선하지만, 훈련 데이터 비율이 높을 때는 전체 배치 옵티마이저와 가중치/활성화 잡음이 없는 모델에서도 일부 일반화가 발생한다. 좋지 않은 하이퍼파라미터 선택은 일반화를 심각하게 제한한다. 표시하지 않았지만, 모든 최적화 방법에서 훈련 정확도는 10^3 - 10^4 업데이트 후 100%에 도달한다. 오른쪽. 다양한 알고리즘 데이터셋에서 10^4 단계 후 달성한 최고 검증 정확도를 3개 시드에 대해 평균한 결과. 직관적으로 더 복잡하고 대칭성이 낮은 연산은 더 높은 데이터 비율에서야 일반화가 일어난다.

그림 1 가운데는 추상 군 S5의 곱셈에서 검증 성능이 처음 99%에 도달할 때까지의 최적화 단계 수 중앙값을 보인다. 데이터 비율 25-30% 부근에서는 훈련 데이터를 1% 줄일 때 일반화까지 걸리는 중앙 시간이 40-50% 증가한다. 검증 정확도 99% 초과에 도달하는 단계 수는 데이터셋 크기가 줄수록 빠르게 증가하지만, 훈련 정확도가 처음 99%에 도달하는 단계 수는 대체로 감소하며 10^3 - 10^4 최적화 단계 범위에 머문다. 네트워크가 일반화한 모든 알고리즘 과제에서 데이터셋 크기 감소에 따라 일반화 도달 최적화 시간이 지수적으로 증가하는 유사한 패턴을 관찰했다.

3.2 다양한 문제에서의 그로킹

부록 A.1.1에 나열한 다양한 이항 연산에 대해, 가능한 모든 식 중 서로 다른 비율을 훈련 데이터셋으로 사용한 세 번의 실행에서 평균 정확도를 측정했다. 결과는 그림 2 오른쪽에 제시했다.

피연산자는 서로 무관한 추상 기호로 제시되므로, 소수 p에 대해 0이 아닌 x, y를 사용하는 $x + y \pmod{p-1}$ 와 $x \cdot y \pmod{p}$ 는 신경망 관점에서 구분할 수 없다. 마찬가지로 $x - y \pmod{p-1}$ 와 $x/y \pmod{p}$ 도 구분할 수 없다. 이는 소수에 대한 모든 0이 아닌 잔여류가 원시근의 거듭제곱으로 표현될 수 있기 때문이다. 이 표현은 mod p-1의 모듈러 덧셈과 mod p의 모듈러 곱셈이 기호 이름을 바꾸는 수준에서 동등함을 보여 준다. 그림 2 오른쪽에서 $x - y$ 와 x/y 가 실제로 일반화에 거의 같은 양의 데이터를 필요로 함을 볼 수 있다.

그림 2 오른쪽의 일부 연산은 피연산자 순서에 대해 대칭적이다($x + y$, $x*y$, $x^2 + y^2$, $x^2 + xy + y^2$). 이런 연산은 밀접하게 관련된 비대칭 연산($x - y$, x/y , $x^2 + xy + y^2 + x$)보다 일반화에 필요한 데이터가 적은 경향이 있다. Transformer는 위치 임베딩을 무시함으로써 피연산자의 대칭 함수를 쉽게 학습할 수 있으므로, 이 효과는 부분적으로 아키텍처 의존적일 수 있다고 본다.

일부 연산(예: $x^3 + xy^2 + y \pmod{97}$)은 데이터 비율을 95%까지 높여도 허용된 최적화 예산 안에서 일반화로 이어지지 않았다. 수렴한 모델은 실제 패턴을 찾지 못한 채 훈련 데이터셋만 사실상 암기했다. 이런 모델에게 데이터는 사실상 무작위와 같다.

[y 가 홀수이면 $x/y \pmod{p}$, 그렇지 않으면 $x - y \pmod{p}$] 연산은 네트워크가 여러 단순 연산의 혼합을 학습해야 한다. 특히 x 의 역할은 짝수 y 와 짝지을 때는 덧셈군의 잔여류로, 홀수 y 와 짝지을 때는 곱셈군의 잔여류로 해석되어야 한다. 이는 군 또는 환 연산으로 깔끔하게 해석되지 않는 연산에서도 일반화가 가능함을 보여 준다.

3.3 절제 실험과 기법

우리는 네트워크가 데이터셋에서 더 잘 일반화하도록 유도할 수 있는 여러 형태의 정규화를 시도했다. 여기서는 특정 데이터셋 S5에서 전체 배치 경사하강법, 확률적 경사하강법, 큰 또는 작은 학습률, 잔차 dropout(Srivastava et al., 2014), 가중치 감쇠(Loshchilov & Hutter, 2017), 경사 잡음(Neelakantan et al., 2015)을 적용한 데이터 효율 곡선을 제시한다. 결과는 그림 2 왼쪽에 있다.

가중치 감쇠를 추가하면 데이터 효율에 매우 큰 효과가 있으며, 다른 대부분의 개입과 비교해 필요한 샘플 수를 절반 이상 줄인다. 네트워크 초기화 지점 쪽으로의 가중치 감쇠도 효과가 있었지만, 원점 쪽으로의 가중치 감쇠만큼은 아니었다. 이는 작은 알고리즘 과제에서는 대략 0에 가까운 가중치가 적합하다는 사전 가정이 가중치 감쇠의 뛰어난 성능 일부를 설명하지만 전부는 아님을 시사한다. 최적화 과정에 잡음을 추가하는 것(예: 미니배치 사용으로 생기는 경사 잡음, 경사 계산 전후 가중치에 적용하는 Gaussian 잡음)도 일반화에 도움이 된다. 이는 그러한 잡음이 일반화가 더 잘되는 평평한 최소점을 찾도록 유도할 수 있다는 생각과 부합한다. 일반화가 일어나려면 학습률을 비교적 좁은 범위, 즉 한 자릿수 규모 안에서 조정해야 했다.

3.4 임베딩의 정성적 시각화

일반화하는 네트워크를 이해하기 위해, 모듈러 덧셈과 S5의 경우에 출력층 행렬을 시각화했다. 그림 3은 행 벡터의 t-SNE 도표를 보여 준다. 일부 네트워크에서는 기저 수학적 대상의 구조가 도표에 뚜렷하게 반영되어 있다. 예를 들어 모듈러 덧셈의 원형 위상은 각 원소에 8을 더해 형성된 “수직선”으로 나타난다. 이 구조는 가중치 감쇠로 최적화한 네트워크에서 더 뚜렷하다.

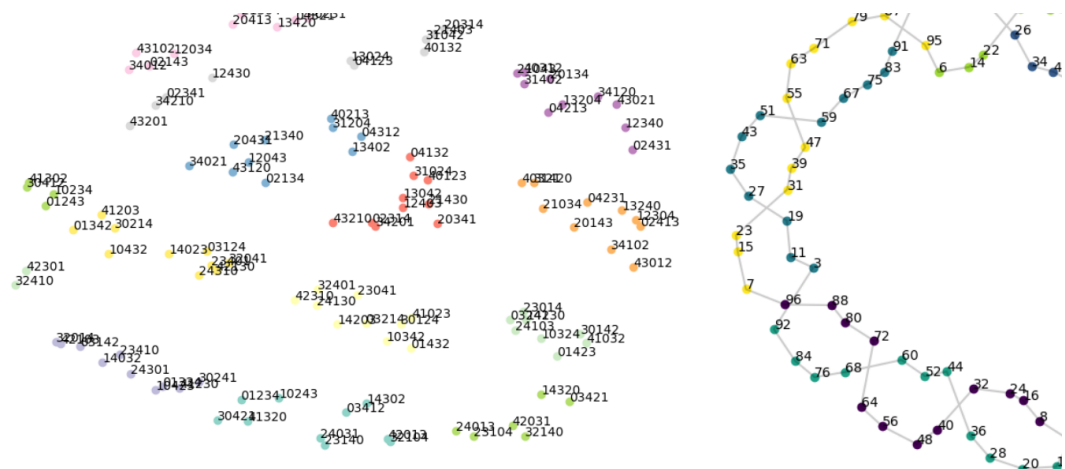


그림 3: 왼쪽. S5로 훈련한 네트워크의 출력층 가중치에 대한 t-SNE 투영. 순열들의 군집이 보이며, 각 군집은 부분군 $\langle(0, 3)(1, 4), (1, 2)(3, 4)\rangle$ 또는 그 켤레 중 하나의 코셋(coset)이다. 오른쪽. 모듈러 덧셈으로 훈련한 네트워크의 출력층 가중치에 대한 t-SNE 투영. 선은 각 원소에 8을 더한 결과를 나타내며, 색은 각 원소의 mod 8 잔여류를 나타낸다.

4 논의

우리가 연구한 소규모 알고리즘 이항 연산표 데이터셋에서는 이중 하강이나 지연된 일반화 같은 효과, 그리고 가중치 감쇠와 같은 개입으로 인한 일반화 개선이 매우 두드러질 수 있음을 보았다. 이는 이러한 데이터셋이 일반화의 여러 측면을 조사하기 좋은 장소가 될 수 있음을 시사한다. 예를 들어 앞으로는 제안된 여러 최소점 평탄도 측정치가 우리의 설정에서 일반화와 상관관계를 보이는지 시험할 계획이다.

또한 신경망의 임베딩 공간을 시각화하면 자연스러운 종류의 구조를 보여 줄 수 있음을 확인했다. 예컨대 모듈러 산술 문제에서 임베딩의 위상은 원 또는 원통인 경향이 있다. 네트워크는 여러 잔여류에 따라 임베딩을 특이한 방식으로 정리하는 경향도 보인다. 이러한 수학적 대상의 성질은 우리에게 익숙하지만, 장차 이런 시각화가 새로운 수학적 대상에 대한 직관을 얻는 유용한 방법이 될 수 있다고 추측한다.

추가로, 일정 성능 수준에 도달하는 데 필요한 최적화 단계 수가 훈련 데이터셋 크기를 줄일수록 빠르게 증가한다는 흥미로운 현상을 기록했다. 이는 적은 데이터에서 성능을 얻기 위해 계산량을 교환하는 방식이므로, 앞으로 이 효과가 다른 데이터셋에도 존재하는지 조사할 필요가 있다.

참고문헌

- Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine learning practice and the bias-variance trade-off. arXiv preprint arXiv:1812.11118, 2018.
- Stephane d'Ascoli, Levent Sagun, and Giulio Biroli. Triple descent and the two kinds of overfitting: Where & why do they appear? arXiv preprint arXiv:2006.03509, 2020.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. arXiv preprint arXiv:1807.03819, 2018.
- Alex Graves. Adaptive computation time for recurrent neural networks. arXiv preprint arXiv:1603.08983, 2016.
- Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. arXiv preprint arXiv:1410.5401, 2014.
- Edward Grefenstette, Karl Moritz Hermann, Mustafa Suleyman, and Phil Blunsom. Learning to transduce with unbounded memory. Advances in neural information processing systems, 28:1828-1836, 2015.
- Sepp Hochreiter and Jurgen Schmidhuber. Flat minima. Neural computation, 9(1):1-42, 1997.
- Yiding Jiang, Behnam Neyshabur, Hossein Mobahi, Dilip Krishnan, and Samy Bengio. Fantastic generalization measures and where to find them. arXiv preprint arXiv:1912.02178, 2019.
- Łukasz Kaiser and Ilya Sutskever. Neural gpus learn algorithms. arXiv preprint arXiv:1511.08228, 2015.
- Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. CoRR, abs/1609.04836, 2016.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.
- Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. arXiv preprint arXiv:1912.02292, 2019.
- Arvind Neelakantan, Luke Vilnis, Quoc V Le, Ilya Sutskever, Łukasz Kaiser, Karol Kurach, and James Martens. Adding gradient noise improves learning for very deep networks. arXiv preprint arXiv:1511.06807, 2015.
- Scott Reed and Nando De Freitas. Neural programmer-interpreters. arXiv preprint arXiv:1511.06279, 2015.
- David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. Analysing mathematical reasoning abilities of neural models. arXiv preprint arXiv:1904.01557, 2019.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. The journal of machine learning research, 15(1):1929-1958, 2014.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. arXiv preprint arXiv:1706.03762, 2017.
- Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. arXiv preprint arXiv:1410.3916, 2014.
- Jason Weston, Antoine Bordes, Sumit Chopra, Alexander M Rush, Bart van Merriënboer, Armand Joulin, and Tomas Mikolov. Towards ai-complete question answering: A set of prerequisite toy tasks. arXiv preprint arXiv:1502.05698, 2015.
- Wojciech Zaremba and Ilya Sutskever. Reinforcement learning neural turing machines-revised. arXiv preprint arXiv:1505.00521, 2015.
- Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. arXiv preprint arXiv:1611.03530, 2016.

부록 A

A.1 추가 실험 세부 사항

A.1.1 이항 연산

다음은 시도한 이항 연산들이다(소수 $p = 97$).

$$x \circ y = x + y \pmod{p}, 0 \leq x, y < p$$

$$x \circ y = x - y \pmod{p}, 0 \leq x, y < p$$

$$x \circ y = x/y \pmod{p}, 0 \leq x < p, 0 < y < p$$

$$x \circ y = [y \text{가 홀수이면 } x/y \pmod{p}, \text{ 그렇지 않으면 } x - y \pmod{p}], 0 \leq x, y < p$$

$$x \circ y = x^2 + y^2 \pmod{p}, 0 \leq x, y < p$$

$$x \circ y = x^2 + xy + y^2 \pmod{p}, 0 \leq x, y < p$$

$$x \circ y = x^2 + xy + y^2 + x \pmod{p}, 0 \leq x, y < p$$

$$x \circ y = x^3 + xy \pmod{p}, 0 \leq x, y < p$$

$$x \circ y = x^3 + xy^2 + y \pmod{p}, 0 \leq x, y < p$$

$$x \circ y = x \cdot y, x, y \in S_5$$

$$x \circ y = x \cdot y \cdot x^{-1}, x, y \in S_5$$

$$x \circ y = x \cdot y \cdot x, x, y \in S_5$$

각 이항 연산에 대해 $h_{xihopihyih}=ihx \circ yi$ 형태의 식 데이터셋을 구성했다. 여기서 hai는 원소 a에 해당하는 토큰을 뜻한다. 각 훈련 실행에서 가능한 모든 식 중 일정 비율을 무작위로 골라 훈련 집합으로 선언했고, 나머지 식은 검증 집합으로 사용했다.

A.1.2 모델과 최적화

인과적 attention masking을 사용하는 표준 decoder-only Transformer(Vaswani et al., 2017)를 훈련했고, 식의 답 부분에서만 손실과 정확도를 계산했다. 모든 실험에서 2개 층, 폭 128, attention head 4개를 갖고 비임베딩 파라미터가 약 4×10^5 개인 Transformer를 사용했다.

모듈러 덧셈과 S5의 곱셈에서 실험을 수행해 최적화 하이퍼파라미터를 조정했다. 최종 구성은 S5에서 확인한 성능과 단순성의 균형을 택했다. 예를 들어 일부 상황에서는 학습률 어닐링이 더 좋았지만, 논문의 실험에서는 사용하지 않았다. 대부분의 실험에서 AdamW 옵티마이저(학습률 10^{-3} , 가중치 감쇠 1, $\beta_1 = 0.9$, $\beta_2 = 0.98$), 처음 10회 업데이트에 대한 선형 학습률 warmup, 미니배치 크기 512 또는 훈련 데이터셋 크기의 절반 중 더 작은 값, 그리고 10^5 회 경사 업데이트의 최적화 예산을 사용했다.

3.3절에서는 그림 2 왼쪽의 읽는 순서대로 다음 변형도 시도했다.

- 전체 배치 Adam 옵티마이저(즉, 전체 훈련 데이터셋 손실의 정확한 경사).
- Adam 옵티마이저.
- 각 파라미터의 업데이트 방향에 Gaussian 잡음을 더한 전체 배치 Adam: $W \leftarrow W + lr \cdot (\Delta W + \epsilon)$, 여기서 ϵ 은 단위 Gaussian에서 샘플링하고 ΔW 는 표준 Adam 가중치 업데이트, lr 은 학습률이다.
- 잔차 dropout 0.1을 추가한 모델의 Adam 옵티마이저.
- 가중치 감쇠 1을 적용한 AdamW 옵티마이저(다른 대부분 실험의 기본 설정).
- 원점 대신 초기화 지점 쪽으로 가중치 감쇠 1을 적용한 AdamW 옵티마이저.
- 학습률 $3 \cdot 10^{-4}$ 의 Adam 옵티마이저.
- 학습률 $3 \cdot 10^{-3}$ 의 Adam 옵티마이저.
- 표준편차 0.01의 Gaussian 가중치 잡음을 사용하는 모델의 Adam 옵티마이저(모델에서 각 파라미터 W 를 $W + 0.01 \cdot \epsilon$ 로 바꾸며, ϵ 은 단위 Gaussian에서 샘플링).

3.1.1절의 실험에서는 완전한 일반화까지 걸리는 시간이 증가하는 양상을 더 잘 포착하기 위해 최적화 예산을 $5 \cdot 10^5$ 단계로 늘렸다. 3.1절의 실험에서는 일반화가 최적화 과정의 얼마나 늦은 시점에 시작될 수 있는지 강조하기 위해 최적화 예산을 10^6 단계로 늘리고, 가중치 감쇠가 없는 Adam 옵티마이저를 사용했다. 3.1.1절 실험은 7개 무작위 시드의 결과를 집계했고, 그 밖에는 데이터셋 크기별 각 실험을 3개 무작위 시드로 반복했다.

A.2 추가 그림

그림 4에는 그림 1의 정확도 곡선에 대응하는 손실 곡선을 제시한다. 그림 5에는 네트워크가 실제로 풀 수 있는 이항 연산표의 예를 보인다.

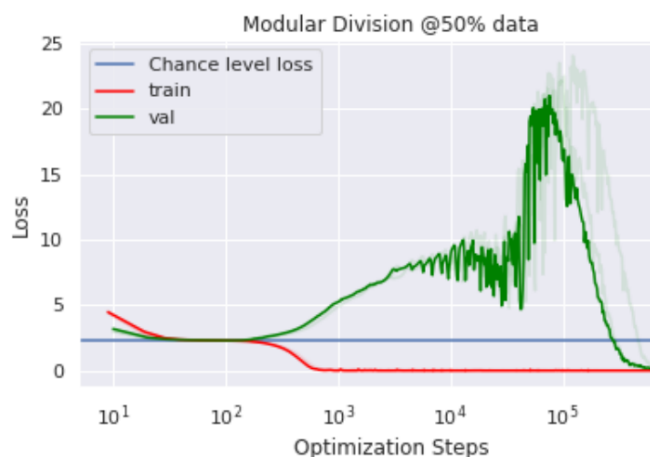


그림 4: 모듈러 나눗셈의 훈련 및 검증 손실 곡선. 검증 손실은 10^2 단계부터 약 10^5 최적화 단계까지 증가한 뒤 두 번째 하강을 시작한다.

A.3 관련 연구

본 논문은 작고 단순한 알고리즘 데이터셋에서의 훈련 및 일반화 동역학을 연구한다. 과거에는 알고리즘 데이터셋을 신경망의 기호적·알고리즘적 추론 능력을 탐색하는 데 사용해 왔다. 예를 들어 무작위로 생성된 수열을 복사, 역순 정렬, 정렬하는 과제와 여러 자리 수의 산술 연산은 sequence-to-sequence 모델의 표준 벤치마크로 쓰였다(Graves et al., 2014; Weston et al., 2014;

Kaiser & Sutskever, 2015; Reed & De Freitas, 2015; Grefenstette et al., 2015; Zaremba & Sutskever, 2015; Graves, 2016; Dehghani et al., 2018). 그러나 이런 연구는 보통 무제한 데이터 환경에서의 성능을 강조하며, 일반화는 흔히 입력 수열 길이에 대해 연구된다. 일부 논문은 알고리즘 과제의 샘플 복잡도를 연구했지만(Reed & De Freitas, 2015), 대체로 아키텍처 선택의 영향에 집중한다. 이와 대조적으로 본 연구는 데이터가 제한된 환경의 일반화 현상에 초점을 두며, 아키텍처에 덜 의존적이라고 생각하는 현상을 강조한다.



그림 5: 네트워크에 제시된 이항 연산 표 중 네트워크가 완벽하게 채울 수 있는 사례. 독자의 편의를 위해 각 기호는 영어, 히브리어 또는 그리스어 알파벳 문자로 나타냈다. 여기서 어떤 연산이 표현되었는지 추측해 볼 수 있다.

bAbI와 같은 알고리즘 생성 추론 데이터셋(Weston et al., 2015)은 데이터 제한 환경의 일반화를 연구하도록 장려한다. 그러나 이런 데이터셋의 대부분 결과는 특정 아키텍처나 훈련 기법의 성능 점 추정치에 초점을 맞추며, 우리의 주요 관심사는 특정 아키텍처가 훈련 데이터를 완전히 암기할 수 있게 된 시점 이후에 일반화가 어떻게 변하는지를 지적하는 데 있다.

Neelakantan et al. (2015)에는 알고리즘 과제에서 “그로킹과 유사한” 학습 곡선이 있으나, 이는 최적화 난이도와 관련된 반면 우리의 현상은 구체적으로 일반화에 관한 것이다.

Saxton et al. (2019)은 산술과 미분 같은 철학적 생성 수학 문제에서 일반화를 연구한다. 그러나 대체로 이런 과제는 우리가 연구한 단순한 이항 연산 문제보다 복잡하여, 본 논문에서 기술하는 현상을 관찰하기에는 적합하지 않다. 그런 과제를 숙달하려면 매우 많은 샘플이 필요하기 때문이다.

Jiang et al. (2019)은 합성곱 신경망에서 일반화 성능을 예측하는 측정치가 무엇인지 알아보기 위해 다수의 일반화 및 복잡도 측정치를 연구했다. 이들은 훈련된 신경망이 파라미터 교란에 얼마나 민감한지 정량화하려는 평탄도 기반 측정치가 가장 예측력이 높다고 보았다. 우리는 본 논문에서 보고한 그로킹 현상이 SGD의 잡음이 최적화를 더 평평하고 단순한, 따라서 일반화가 더 잘되는 해로 밀어 주기 때문일 수 있다고 추측했으며, 앞으로 이들 측정치가 그로킹을 예측하는지 조사하고자 한다.

Zhang et al. (2016)은 딥러닝에서 통상 사용되는 크기의 신경망이 임의의 훈련 데이터셋을 보간할 수 있으면서도, 적절한 최적화 절차로 의미 있는 레이블을 사용해 훈련하면 일반화한다는 점을 보였다. 본 연구는 신경망이 작은 알고리즘 훈련 데이터셋을 일반화 없이 보간할 수 있지만, SGD로 더 오래 훈련하면 일반화를 시작한다는 관련 현상을 보인다.

Nakkiran et al. (2019)과 Belkin et al. (2018)은 모델 및 최적화 절차의 용량 함수로서 나타나는 손실의 이중 하강에 초점을 둔다. 이들은 고전적인 U자형 검증 손실 곡선 뒤에, 모든 훈련 데이터를 보간하는 데 필요한 최소 용량 부근에서 시작하는 두 번째 손실 하강이 일부 설정(신경망 훈련 포함)에서 이어짐을 보인다. 우리도 일부 실험에서 훈련량의 함수로 검증 손실(정확도는 아님)이 두 번째 하강을 보이는 것을 관찰하며, 이는 훈련 데이터를 보간한 뒤에 발생한다. 다만 우리가 기술한 현상은 두 번째 손실 하강이 훈련 손실이 처음 매우 작아진 때보다 훨씬 뒤(일부 실험에서는 수만 에포크 뒤)에 나타나고, 정확도에서 비단조적 거동을 관찰하지 않기 때문에 기존의 이중 하강 현상과 구별될 수 있다고 본다. 또한 우리가 연구한 소규모 알고리즘 데이터셋은 Nakkiran et al. (2019)이 다룬 자연 데이터셋보다 미묘한 일반화 현상을 연구하기에 더 작고 다루기 쉬운 장을 제공한다.

A.4 여러 이상치를 암기하는 조건에서의 일반화

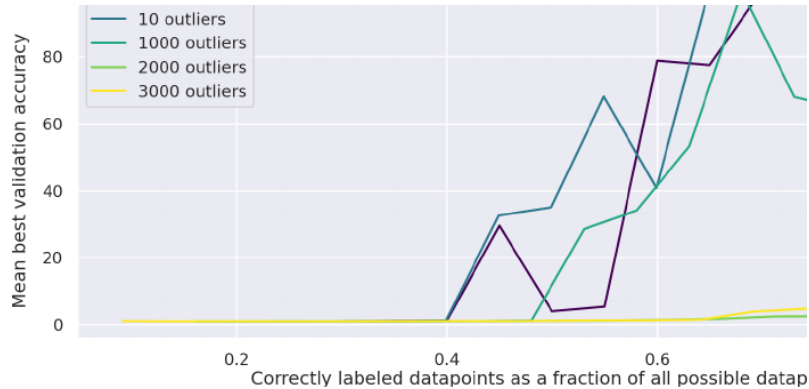


그림 6: 훈련 데이터에 $k \in [0, 10, 100, 1000, 2000, 3000]$ 개의 이상치(무작위 레이블을 가진 예시)를 도입했을 때의 데이터 효율 효과. 적은 수의 이상치는 일반화 성능에 눈에 띄는 영향을 주지 않지만, 많은 이상치는 성능을 크게 저해한다.

이 절에서는 이항 연산의 수정 버전에 대해, 훈련 데이터셋에 k 개의 이상치를 도입한 데이터 효율 곡선을 보인다. 구체적으로는 실험 시작 시 훈련 집합에서 k 개의 식을 무작위로 표본 추출하고, 그 답을 훈련 데이터에서 무작위로 뽑은 다른 k 개 식의 답으로 교체한다. 훈련 데이터의 나머지 식과 검증 데이터의 모든 식은 그대로 둔다.

이 상황에서는 몇 가지 시나리오를 생각해 볼 수 있다. 이전과 같이 최적화되고 정규화된 신경망 모델 부류가 이런 “잡음” 데이터셋을 보간하기에 충분히 크지 않다면, 절차가 잘 일반화하면서 훈련 데이터의 잡음을 제거하는 해로 수렴할 수 있다. 즉, 이상치 식 $a, b \rightarrow c' (c' \neq c)$ 에 대해서도 답으로 $c = a \circ b$ 를 예측할 수 있다. 반대 극단에서는 최적화 절차가 데이터를 보간하는 네트워크를 찾지만, 단순한 함수에 훈련 데이터에 인코딩된 k 개의 예외를 더한, 이전보다 훨씬 복잡한 함수를 표현해야 하므로 결과 모델이 일반화하지 못할 수 있다.

실험에서는 첫 번째 가능성이 일어나지 않았다. 모든 실험이 어느 시점에서는 훈련 정확도 100%에 도달했으며, 그 시점은 이상치 수 k 를 바꿔도 크게 영향을 받지 않았다. 두 번째 현상은 일정 범위의 훈련 데이터 비율과 이상치 수 k 에서 나타났다. k 가 증가할수록 최적화 절차가 일반화하는 모델로 수렴하는 훈련 데이터 비율의 범위가 줄어든다. 그러나 적은 수의 이상치(최대 1000개)를 도입한 효과는 매우 두드러지지는 않았다(그림 6). 이는 네트워크와 최적화 절차의 용량이 훈련 데이터의 모든 레이블을 암기하는 데 필요한 용량을 훨씬 넘으며, 일반화가 아예 일어난다는 사실에는 자명하지 않은 설명이 필요함을 뒷받침하는 추가 증거로 해석한다.

A.5 일반화 측정치

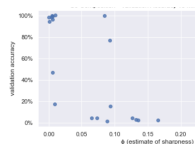


그림 7: S5 합성 목표로 훈련한 네트워크는 손실 지형의 비교적 평평한 영역에서만 그로킹하는 것으로 보인다.

본 논문에서 제시한 소규모 알고리즘 데이터셋에서는 통상적 일반화 측정치의 예측력을 살펴볼 가치가 있다. 예비 조사에서 S5 합성 목표를 고정 단계 동안 여러 초기화 시드로 훈련해 약 절반이 높은 검증 정확도를 얻도록 한 뒤, Keskar et al. (2016)의 방법으로 각 최소점의 sharpness(Hochreiter & Schmidhuber, 1997) 근사값 ϕ 를 계산했다. 검증 정확도와 ϕ 의 Spearman 상관계수는 $-0.79548 (p < 0.000014)$ 이었다.